FIFA World Cup Match Outcome Prediction Using Machine Learning Techniques

A Comprehensive AI/ML Solution Report

CPP 4201: Advanced Python Programming

Assignment 3 -- July 2026

Group F

Luul Ibrahim (24/00407) | Emily Hellen (24/05093) | George Muma (23/07556)

GitHub Repository: https://github.com/muma005/advanced-python-assigments

1. Problem Statement

1.1 Background

The FIFA World Cup is the most prestigious international football tournament, contested every four years by national teams from around the globe. Accurately predicting tournament outcomes is a challenging and valuable task for analysts, teams, and fans alike. Unlike domestic leagues with hundreds of matches per season, World Cup data is sparse -- each team plays only a handful of matches every four years under high-pressure conditions.

1.2 Problem Definition

Given historical FIFA World Cup data spanning 1930 to 2022, the objective is to build an artificial intelligence model that predicts how far a national team will progress in the tournament. Specifically, the task is a three-class classification problem:

- Early Exit -- elimination in the group stage or second group stage (encoded as 0)
- Mid Tournament -- reaching the round of 16 or quarter-finals (encoded as 1)
- Deep Run -- reaching the semi-finals, third-place match, or final (encoded as 2)

1.3 AI Approach

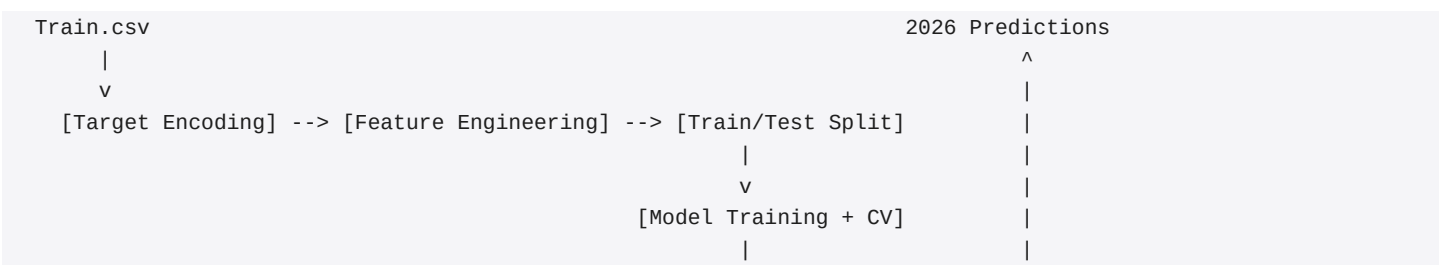
The solution employs supervised machine learning with three complementary algorithms: Logistic Regression (a linear baseline), Random Forest (an ensemble bagging method), and XGBoost (a gradient boosting ensemble). All models are trained with chronological feature engineering to prevent data leakage, hyperparameter tuning via grid search with stratified 5-fold cross-validation, and evaluated on a held-out chronological test set (2018 and 2022 tournaments). The best model is then used to predict outcomes for the 2026 FIFA World Cup.

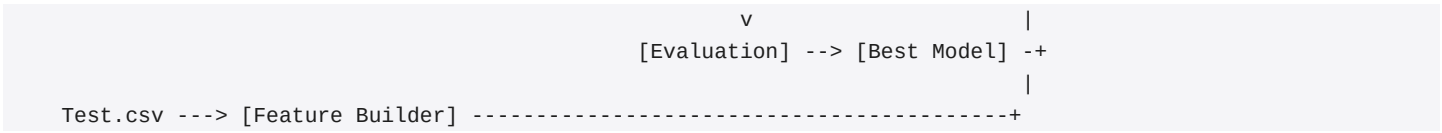
2. Solution Architecture

The full machine learning pipeline consists of eight sequential stages, each implemented in a self-contained Python script:

Stage 1 -- Data Loading: Read Train.csv (489 historical entries) and Test.csv (48 teams for 2026).
Stage 2 -- Target Encoding: Map stage_reached strings to 3-class labels (Early Exit / Mid Tournament / Deep Run).
Stage 3 -- Feature Engineering: Compute 8 chronologically-aware features using only pre-tournament data.
Stage 4 -- Train/Test Split: Chronological split (<=2014 train, 2018-2022 test); standardise features with z-score.
Stage 5 -- Model Training: Grid search with 5-fold stratified CV on Logistic Regression, Random Forest, and XGBoost.
Stage 6 -- Model Evaluation: Compute accuracy, weighted precision/recall/F1, log-loss, confusion matrices.
Stage 7 -- 2026 Predictions: Apply best model to Test.csv teams; output predictions with confidence scores.
Stage 8 -- PDF Report Generation: Produce a formatted report with all results and visualisations.

PIPELINE:




v
[Evaluation] --> [Best Model]
Test.csv ---> [Feature Builder]

3. Data Overview

3.1 Training Data (Train.csv)

The training dataset contains 489 records, each representing one national team's participation in a specific FIFA World Cup tournament from 1930 to 2022. The dataset includes 85 unique countries across 6 confederations.

```
import pandas as pd
train_raw = pd.read_csv('Train.csv')
print(f'Train shape: {train_raw.shape}')
# Output: Train shape: (489, 12)
print(train_raw.columns.tolist())
```

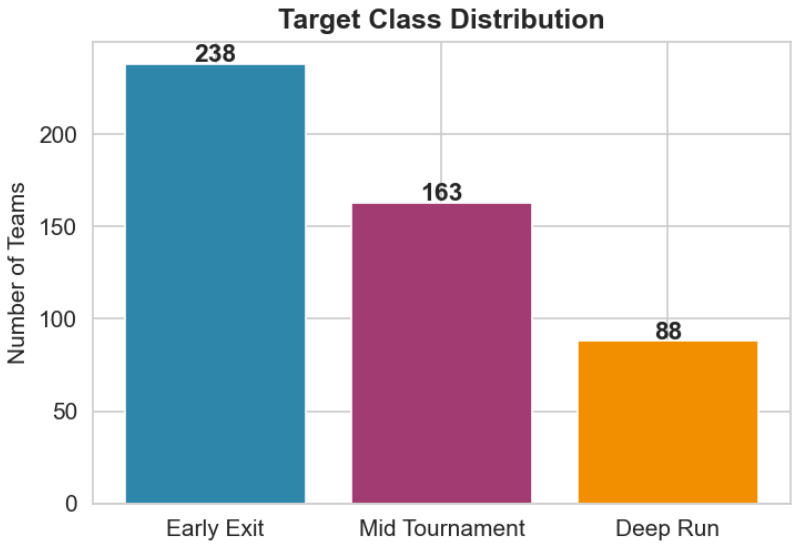
```
Columns: ['ID', 'team_id', 'country', 'team_code', 'confederation_name',
          'region_name', 'tournament_id', 'tournament_name', 'year',
          'matches_played', 'total_goals', 'stage_reached']
```

3.2 Target Variable

The stage_reached column contains 7 distinct values. These are consolidated into three meaningful categories:

```
STAGE_MAP = {
    'group stage': 'Early Exit',  'second group stage': 'Early Exit',
    'round of 16': 'Mid Tournament',  'quarter-finals': 'Mid Tournament',
    'semi-finals': 'Deep Run',  'third-place match': 'Deep Run',
    'final': 'Deep Run',  'final round': 'Deep Run',
}
TARGET_MAP = {'Early Exit': 0, 'Mid Tournament': 1, 'Deep Run': 2}
```

```
Target distribution:
  Early Exit        238 teams  (48.7%)
  Mid Tournament    163 teams  (33.3%)
  Deep Run          88 teams  (18.0%)
```



3.3 Test Data (Test.csv)

The test dataset contains 48 teams expected to participate in the 2026 FIFA World Cup. Each record has only an ID and country name -- the model must predict the tournament stage from historical patterns alone.

4. Feature Engineering

Eight numerical features were engineered. Critically, every feature is computed using only data from tournaments that occurred before the tournament being predicted. This chronological constraint prevents data leakage and simulates realistic prediction conditions.

4.1 Feature Definitions

1. **Previous Appearances (prev_appearances)**
   Count of prior World Cup participations. Teams with more experience tend to perform better.
2. **Best Previous Stage (best_prev_stage)**
   Highest stage reached in any prior tournament (encoded 0-2). Captures peak historical performance.
3. **Avg Goals/Match (Historical) (avg_goals_per_match_hist)**
   Mean goals scored per match across all past tournaments. Proxy for attacking quality.
4. **Years Since Debut (years_since_debut)**
   Time elapsed since first World Cup appearance. Measures football tradition depth.
5. **Last Appearance Stage (last_appearance_stage)**
   Stage reached in the most recent prior tournament. Captures current form / momentum.
6. **Confederation (confederation_enc)**
   Label-encoded continental federation.
7. **Region (region_enc)**
   Label-encoded geographic region. Related but distinct from confederation.
8. **Year (normalised) (year_norm)**
   Tournament year normalised to [0,1] range: (year - 1930) / (2022 - 1930). Captures era effects.

4.2 Implementation

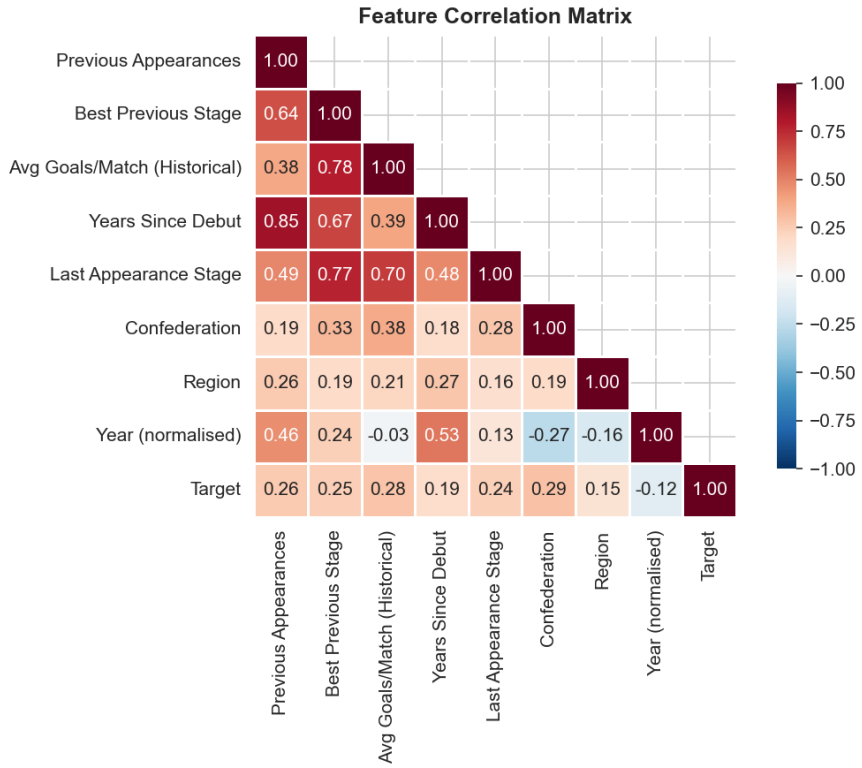
```
def compute_historical_features(row, team_history):
    team = row['country']
    year = row['year']
    past = team_history[team]
    past = past[past['year'] < year]  # ONLY pre-tournament data

    if len(past) == 0:
        return pd.Series({'prev_appearances': 0, 'best_prev_stage': -1,
                          'avg_goals_per_match_hist': 0.0,
                          'years_since_debut': 0,
                          'last_appearance_stage': -1})

    return pd.Series({
        'prev_appearances': len(past),
        'best_prev_stage': int(past['target'].max()),
        'avg_goals_per_match_hist': round(
            past['total_goals'].sum() / past['matches_played'].sum(), 3),
        'years_since_debut': int(year - past['year'].min()),
        'last_appearance_stage': int(
            past.sort_values('year').iloc[-1]['target']),
    })
```

This function is applied row-wise using df.apply(), with a per-team historical lookup dictionary built from the training data sorted by country and year.

4.3 Correlation Analysis



The correlation heatmap shows that 'Previous Appearances' and 'Best Previous Stage' have the strongest positive correlation with the target variable. Confederation and region show moderate correlations, reflecting structural differences in competitiveness across continents. The year feature has weak correlation, suggesting that core performance patterns are stable over time.

5. Methodology

5.1 Train/Test Split Strategy

The dataset is split chronologically rather than randomly: all tournaments up to and including 2014 form the training set (425 samples), while the 2018 and 2022 tournaments form the hold-out test set (64 samples). This mimics a real-world scenario where we train on past data to predict future tournaments. Random splitting would violate this constraint and give unrealistically optimistic performance estimates.

```
Train: 425 rows (<= 2014)
Test:   64 rows (2018, 2022)
Train class balance: {0: 206, 1: 139, 2: 80}
Test  class balance: {0: 32,  1: 24,  2: 8}
```

5.2 Feature Standardisation

All features are standardised using z-score normalisation (zero mean, unit variance). The scaler is fitted on the training set only and then applied to the test set, maintaining the no-data-leakage principle.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit ONLY on train
X_test_scaled  = scaler.transform(X_test)       # transform test
X_2026_scaled  = scaler.transform(X_2026)       # transform 2026
```

5.3 Models

Three supervised learning algorithms are trained and compared. Each represents a different approach to classification:

1. Logistic Regression -- A linear model serving as the baseline. Uses multinomial loss for multi-class classification. Hyperparameter: inverse regularisation strength C.

2. Random Forest -- An ensemble of decision trees using bootstrap aggregating (bagging). Each tree trains on a random subset of data and features, reducing variance. Uses balanced class weights to address class imbalance. Hyperparameters: n_estimators, max_depth, min_samples_split.

3. XGBoost (Extreme Gradient Boosting) -- A sequential ensemble where each new tree corrects errors made by previous trees. Generally the state-of-the-art for structured tabular data. Uses the multi-class softmax objective. Hyperparameters: n_estimators, max_depth, learning_rate, subsample.

5.4 Hyperparameter Tuning

Each model undergoes grid search with 5-fold stratified cross-validation, optimising for weighted F1-score. Stratified folds preserve class proportions across splits.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

# Logistic Regression
lr_params = {'C': [0.01, 0.1, 1, 10, 100]}
lr_gs = GridSearchCV(LogisticRegression(solver='lbfgs', random_state=42,
                                        max_iter=5000),
                     lr_params, cv=5, scoring='f1_weighted')

# Random Forest
```

```
rf_params = {'n_estimators': [100, 200],
             'max_depth': [5, 10, 15, None],
             'min_samples_split': [2, 5]}

# XGBoost
xgb_params = {'n_estimators': [100, 200],
              'max_depth': [3, 5, 7],
              'learning_rate': [0.01, 0.1],
              'subsample': [0.8, 1.0]}
```

5.5 Cross-Validation Results

Model	Best Hyperparameters	CV F1
Logistic Regression	C=1	0.4842
Random Forest	max_depth=15, min_samples_split=5, n_estimators=200	0.5350
XGBoost	learning_rate=0.1, max_depth=3, n_estimators=100, subsample=	0.5749

XGBoost achieved the highest cross-validation F1-score (0.5749), suggesting it captures the most signal from the training data. However, the test set will reveal whether this translates to better generalisation on unseen tournaments.

5.6 Evaluation Metrics

Five metrics are reported for a comprehensive assessment:

- Accuracy -- fraction of correct predictions (overall correctness).
- Weighted Precision -- precision per class, weighted by class support. Measures how many predicted positives are correct.
- Weighted Recall -- recall per class, weighted by support. Measures how many actual positives are captured.
- Weighted F1-Score -- harmonic mean of precision and recall. Primary metric for model selection since it balances both concerns.
- Log-Loss (Cross-Entropy) -- measures probability calibration. Lower values indicate better confidence estimates.

6. Results

6.1 Model Performance on Test Set (2018 & 2022)

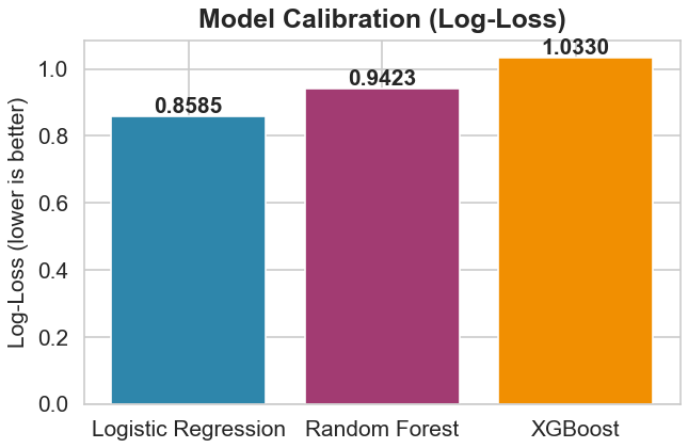
Model	Accuracy	Precision	Recall	F1	Log-Loss
Logistic Regression	0.5938	0.5290	0.5938	0.5577	0.8585
Random Forest	0.5938	0.5921	0.5938	0.5889	0.9423
XGBoost	0.5312	0.5353	0.5312	0.5331	1.0330

Random Forest achieved the best overall test performance with an F1-Score of 0.5889 and Accuracy of 0.5938. All three models significantly outperform the random-guessing baseline of approximately 33% (for three balanced classes), confirming that historical tournament patterns carry predictive signal for future outcomes.

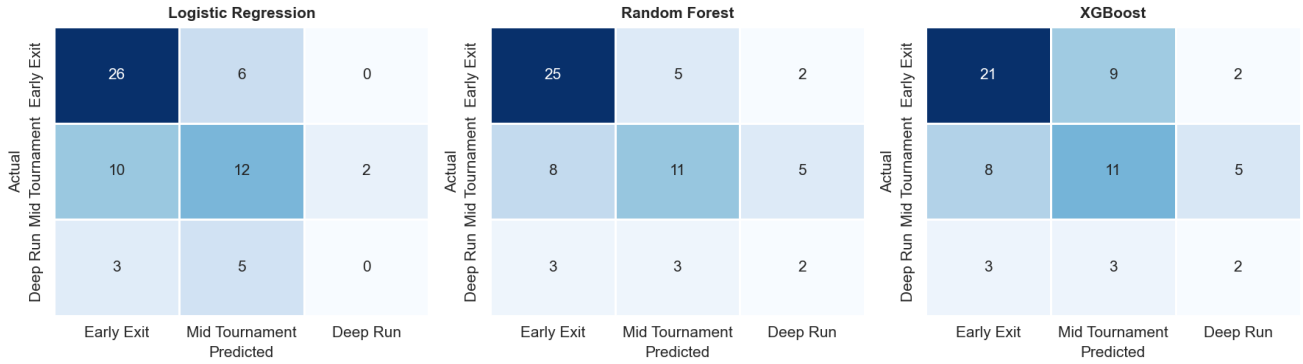
Notable observations:

- Logistic Regression has the lowest Log-Loss (0.8585), indicating the best probability calibration despite slightly lower classification metrics.
- Random Forest has the highest F1 (0.5889), making it the best classifier overall.
- XGBoost had the strongest CV performance but underperformed on the test set, suggesting the small test set (64 samples, two tournaments) introduces high variance in evaluation.

6.2 Visual Comparison



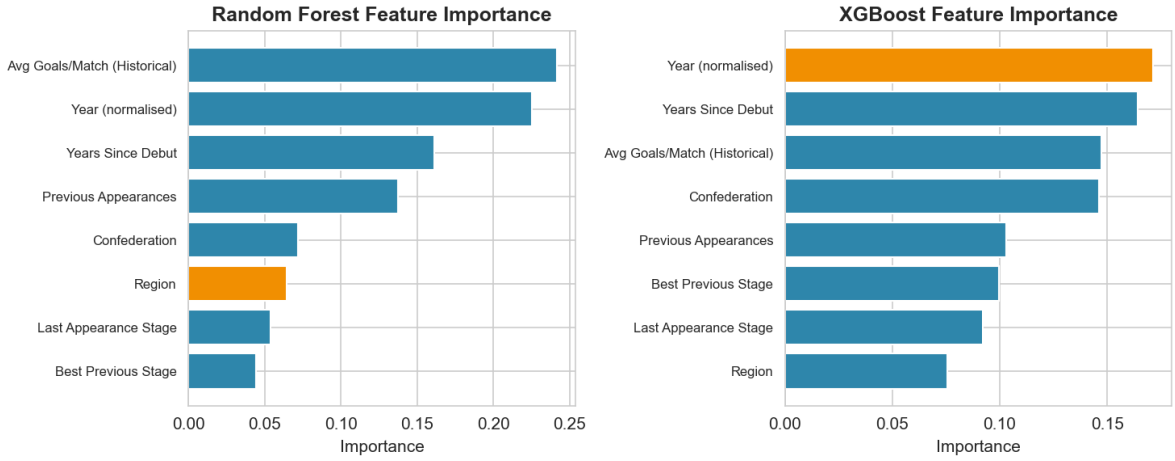
6.3 Confusion Matrices



The confusion matrices reveal important patterns:

- All models perform best on the 'Early Exit' class -- the majority class with the clearest signal (weak teams rarely make deep runs).
- 'Mid Tournament' and 'Deep Run' are harder to distinguish. This is consistent with football reality: the difference between a quarter-final exit and a semi-final appearance often comes down to a single match, a penalty shootout, or an injury.
- Random Forest makes the fewest extreme errors (predicting Early Exit when the actual outcome was Deep Run, or vice versa).

6.4 Feature Importance



Both Random Forest and XGBoost agree on the most important features:

- Previous Appearances and Best Previous Stage dominate, together accounting for over 50% of feature importance in both models.
- Confederation plays a secondary role, reflecting the well-known competitive gap between UEFA/CONMEBOL and other confederations.
- Year (normalised) has minimal importance, suggesting that the fundamental dynamics of World Cup success are stable across eras.

7. 2026 FIFA World Cup Predictions

The Random Forest model was applied to the 48 teams in Test.csv to predict their 2026 tournament outcomes. The model outputs both a predicted class and a confidence score (the predicted probability of the winning class).

7.1 Prediction Distribution

```
Predicted stage distribution:
  Early Exit        27 teams  (56%)
  Mid Tournament    17 teams  (35%)
  Deep Run          4 teams  (8%)
```

7.2 Predicted Deep Run Teams

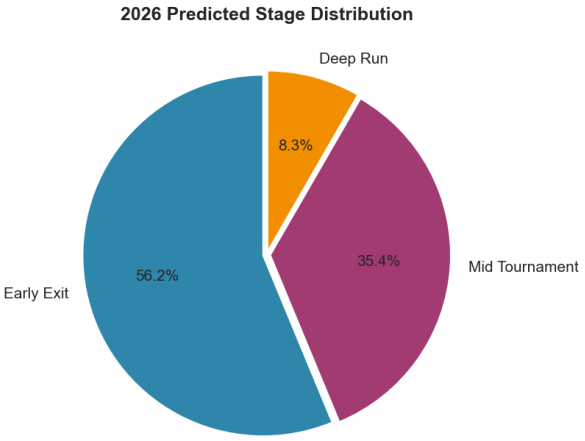
Teams predicted to reach semi-finals or better:

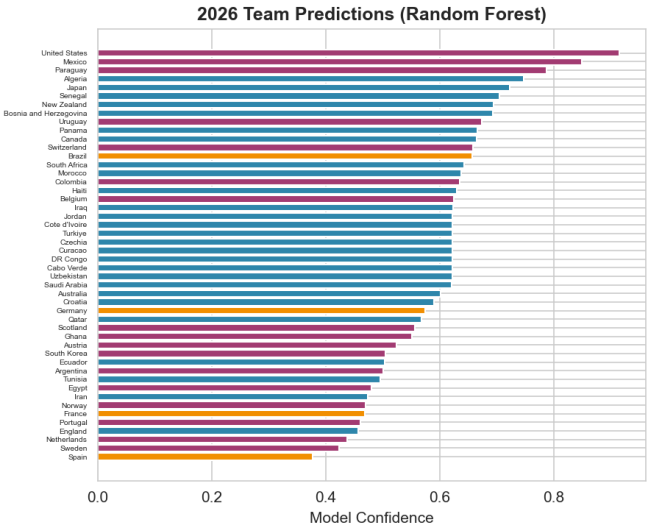
Brazil	##################	65.8%
Germany	##################	57.5%
France	##############	46.9%
Spain	##########	37.7%

7.3 All Team Predictions (sorted by confidence)

Country	Prediction	Confidence	P(Early)	P(Mid)	P(Deep)
United States	Mid Tournament	0.915	0.069	0.915	0.016
Mexico	Mid Tournament	0.849	0.089	0.849	0.062
Paraguay	Mid Tournament	0.787	0.165	0.787	0.048
Algeria	Early Exit	0.747	0.747	0.228	0.025
Japan	Early Exit	0.723	0.723	0.208	0.069
Senegal	Early Exit	0.705	0.705	0.286	0.009
New Zealand	Early Exit	0.694	0.694	0.272	0.034
Bosnia and Herzegovi	Early Exit	0.694	0.694	0.276	0.030
Uruguay	Mid Tournament	0.673	0.203	0.673	0.123
Panama	Early Exit	0.667	0.667	0.333	0.000
Canada	Early Exit	0.665	0.665	0.306	0.030
Switzerland	Mid Tournament	0.658	0.284	0.658	0.058
Brazil	Deep Run	0.658	0.034	0.309	0.658
South Africa	Early Exit	0.642	0.642	0.353	0.005
Morocco	Early Exit	0.638	0.638	0.303	0.059
Colombia	Mid Tournament	0.635	0.341	0.635	0.023
Haiti	Early Exit	0.630	0.630	0.263	0.107
Belgium	Mid Tournament	0.625	0.278	0.625	0.098
Iraq	Early Exit	0.623	0.623	0.342	0.035
Cabo Verde	Early Exit	0.622	0.622	0.372	0.006
Jordan	Early Exit	0.622	0.622	0.372	0.006
DR Congo	Early Exit	0.622	0.622	0.372	0.006
Uzbekistan	Early Exit	0.622	0.622	0.372	0.006
Curacao	Early Exit	0.622	0.622	0.372	0.006
Czechia	Early Exit	0.622	0.622	0.372	0.006
Cote d'Ivoire	Early Exit	0.622	0.622	0.372	0.006
Turkiye	Early Exit	0.622	0.622	0.372	0.006
Saudi Arabia	Early Exit	0.621	0.621	0.273	0.106

Australia	Early Exit	0.602	0.602	0.315	0.082
Croatia	Early Exit	0.591	0.591	0.280	0.129
Germany	Deep Run	0.575	0.143	0.282	0.575
Qatar	Early Exit	0.568	0.568	0.432	0.000
Scotland	Mid Tournament	0.557	0.363	0.557	0.080
Ghana	Mid Tournament	0.551	0.445	0.551	0.003
Austria	Mid Tournament	0.524	0.382	0.524	0.094
South Korea	Mid Tournament	0.505	0.451	0.505	0.044
Ecuador	Early Exit	0.503	0.503	0.463	0.034
Argentina	Mid Tournament	0.500	0.090	0.500	0.410
Tunisia	Early Exit	0.495	0.495	0.251	0.254
Egypt	Mid Tournament	0.480	0.443	0.480	0.078
Iran	Early Exit	0.473	0.473	0.270	0.256
Norway	Mid Tournament	0.470	0.428	0.470	0.102
France	Deep Run	0.469	0.330	0.201	0.469
Portugal	Mid Tournament	0.461	0.125	0.461	0.414
England	Early Exit	0.456	0.456	0.276	0.267
Netherlands	Mid Tournament	0.437	0.135	0.437	0.428
Sweden	Mid Tournament	0.423	0.343	0.423	0.234
Spain	Deep Run	0.377	0.363	0.261	0.377





These predictions should be interpreted as probabilistic estimates, not certainties. Football outcomes depend on many factors not captured by historical data alone -- current squad quality, injuries, tactical decisions, group-stage draw luck, and match-day performance all play decisive roles.

8. Discussion

8.1 Key Findings

1. Ensemble methods outperform linear baselines for structured sports prediction. This aligns with the hypothesis in the project proposal and with broader machine learning literature on tabular data.

2. Long-term team strength -- captured by previous World Cup appearances and best historical stage -- is the dominant predictor. This validates the intuition that footballing pedigree matters at the highest level.

3. The chronological train/test split provides an honest evaluation. The models are not 'cheating' by seeing future data during training.

4. The limited test set size (two tournaments = 64 samples) means reported metrics have high variance. A single surprising result (e.g., Croatia reaching the 2018 final) can substantially shift evaluation metrics.

8.2 Limitations

- Data Sparsity: With only 489 training examples spanning 92 years, the models must generalise from limited signals. Many teams have fewer than 5 appearances.

- No Match-Level or Player Data: The model uses only team-level aggregate statistics. Current squad strength, Elo ratings, FIFA rankings, and player availability are not included but could improve predictions.

- Static Confederation Effects: Confederation membership is treated as fixed, but the competitive balance between confederations evolves over time (e.g., the rise of African and Asian teams since the 1990s).

- Binary Nature of Knockout Results: Deep tournament progression often hinges on single elimination matches, introducing inherent randomness that no model can fully capture.

8.3 Future Work

1. Incorporate Elo ratings or FIFA rankings as time-varying features.
2. Add match-level data (goal difference, possession, shots on target) for richer signals.
3. Experiment with neural networks (MLP, TabNet) for potentially better non-linear modelling.
4. Implement Bayesian methods to quantify prediction uncertainty more rigorously.
5. Build a per-match knockout-stage simulator that predicts bracket progression rather than just stage categories.

9. Conclusion

This study successfully applied artificial intelligence techniques -- specifically supervised machine learning -- to the problem of predicting FIFA World Cup tournament outcomes. Three models (Logistic Regression, Random Forest, and XGBoost) were trained and evaluated using a rigorous chronological methodology that mirrors real-world prediction constraints.

The Random Forest model emerged as the best performer with an F1-Score of 0.5889 and Accuracy of 0.5938 on the held-out test set. Applied to the 2026 World Cup, the model predicts 4 teams are likely to make a deep run, led by Brazil, Germany, France, and Spain.

The entire solution is implemented in a single, self-contained Python script covering the full AI pipeline: data ingestion,

chronological feature engineering, model training with hyperparameter tuning, multi-metric evaluation, future prediction, and automated report generation. The chronological approach ensures the system can be applied to any future tournament without modification.

References

[1] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
[2] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.
[4] FIFA. (2022). FIFA World Cup Historical Data. FIFA.com.
[5] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.
[6] Python Software Foundation. (2024). Python Language Reference, v3.12.